Table of Content

Summary Of Responsibility 2
1. Introduction 3
2.Agile Project Management 3
2.1 First Phase: Outline Planning and Architectural Design 3
2.1.1 Project Planning 3
2.1.2 Project scheduling 3
2.1.3 Staff Management 4
2.1.4 The Design of Software Structure 4
2.2 Second Phase: The Sprint Cycle 4
2.2.1 Assess (Product backlog) 4
2.3 Third Phase: The Project Closure 5
2.4 Risk Management 5
2.5 Quality Management 5
3.Requirements 5
3.1Applying the Requirements Finding Technique 5
3.1.1 Background reading 5
3.1.2 Interviewing 5
3.1.3 Observation 5
3.1.4 Questionnaires 6
3.1.5 Requirements 6
3.2 User Story 6
3.2.1 Epics and User Stories 6
3.2.2 Priorities of User Stories 6
3.2.3 Estimation of User Stories 6
3.2.4 Iteration Planning 6
3.2.5 Adaption to Changes 6
3.2.6 Prototyping 6
4 Analysis and Design 7
4.1 Class Analysis 7
4.1.1 Entity, Boundary and Control Classes identification 7
4.1.2 Reusability 7
4.1.3 Adaptability to Changes 7
4.2 Design of Software 8
4.3 Design Principles 8
4.3.1 Single Responsibility Principle (SRP) 8
4.3.2 Open-Closed Principle (OCP) 8
4.3.3 Don't Repeat Yourself (DRY) 9
4.3.4 Dependency Inversion Principle (DIP) 10
4.3.5 Liskov substitution principle (LSP) 10
5.Implementation and testing 10
5.1 Implementation Strategy and Iteration Plan 10
5.1.1 Implementation Strategy 10
5.1.2 Iteration Plan 10
5.2 Testing （Strategy and Technique） 10
5.2.1 Strategy 10
5.2.2 Techniques 11
5.3 The Using of TDD 12
6. Reference 13
Appendix A:Iteration plan 14
Appendix B:Prototyping 15
Appendix C:UML components 17
Appendix D:Black Box Test with Screenshots 19
Appendix E:Whit Box Test 30
Appendix F:TDD 31

1. Introduction

This report shows our process of developing this system: British Airways check-in system. The system is developed by agile method. In order to develop this system, we first meet to discuss specific requirements. We divided the system into several separate interfaces. As for the user interface, it can be used to select seats, meals, modify choices, select additional services and View flight information. Then we write java code through iterative development. Finally, we improved the user interface and generated some documents, such as user manuals. We hope that the system can help each passenger successfully complete the flight.

2.Agile Project Management

In order to use a high-quality check-in system to meet the needs of customers under given constraints, our team adopted agile methods in the whole project. More specifically, we use agile methods, such as project planning, project scheduling, and risk management.

2.1 First Phase: Outline Planning and Architectural Design

2.1.1 Project Planning

At the very beginning of this project, We quickly conducted several team meetings and established a methodology for applying Agile Project Management. We set deadlines and iteration cycles for product development. Requirements were then identified and a User Story was written, and milestones were specified, such as prototypes, UML diagrams, and programs after each iteration. And, after each iteration, we revisit our plans, requirements, schedule, deliverers, etc. At the same time, we split into two teams, one for front-end and one for back-end development.

2.1.2 Project scheduling

After writing the User Story and forming the acceptance criteria, we divided the development of the application into several tasks, which were developed simultaneously on the front and back ends. We hold weekly full team meetings for both teams where we synchronize progress and set tasks and targets for the next development cycle.

Task Number	Task Name	Duration	Dependency
T1	Input check	7	
T2	Check-in	21	T1
T3	Select Meal	7	T2
T4	Select Seat	14	T2
T5	Special Service	21	T3,T4(M1)
T6	Print Boarding pass	14	T5 (M2)
T7	Usability Optimisation	14	T6
T8	Payment	14	T5 (M3)
T9	Scalability	14	T8(M4)

Figure 1 Task Table

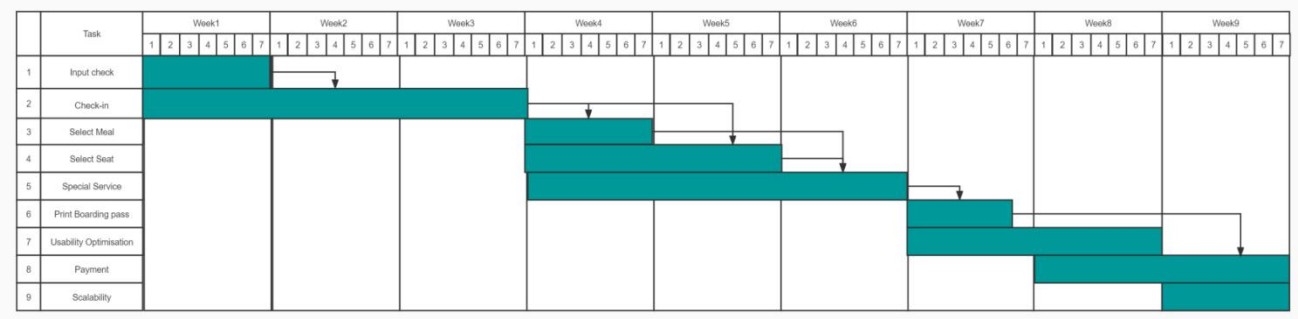


Figure 2 Gantt chart

2.1.3 Staff Management

We split the team of six in half into two teams to work on the front-end and back-end programs. We keep on track through regular meetings

Front-end	Yuqi Liu
	Xiangrui Meng
	Xinlei Ge
Back-end	Junfeng Jin
	Huisi Yang
	Yian Li

Figure 3 Staff Management

2.1.4 The Design of Software Structure

Our airport kiosk software is divided into two main parts - the GUI, which interacts with the user, and the back-end program, which stores and processes the data.
In the front-end application, the user can interact with the GUI to select different login methods and then enter personal information to log in, followed by a series of procedures such as seat selection, meal selection, payment, printing boarding pass and luggage tag, etc. The user can modify their choices or exit the system at any time.
In the back-end application, passenger and flight information is stored in a CSV file and the software operates on the corresponding passengers by creating Customer objects and other objects of control classes. This includes logging in, checking information, selecting seats, choosing meals, paying for meals, etc.

2.2 Second Phase: The Sprint Cycle

2.2.1 Assess (Product backlog)

At the beginning of our project cycle, we carried out user stories and user requirements analysis, which led to the product backlog. The product backlog includes story id, story name, description, priority, acceptance criteria,etc. for completion. In each cycle we have designed tasks that need to be coordinated or iterated to ensure that the project is feasible and reasonable. We also apply the MoSCoW method[1] in DSNM to rank the importance of our software features. Details of our product backlog can be found in the appendix files.

2.2.2 Select (Requirements)

At the beginning of each iteration cycle, we hold a fully attended group meeting to discuss the product features that need to be completed in the next iteration. A selection of the most urgent (most important) existing tasks in the product backlog are chosen as targets for the next iteration. The requirements and design of these approaches are discussed in detail step by step, with each team member being assigned a similar amount of tasks.

2.2.3 Develop (Java coding)

After refining these stories into smaller ones and discussing the implementation of these features in detail, we started the formal coding task. Firstly, we standardized the code specification and the precautions to take, while splitting the team into front-end and back-end sections, and we maintained close communication within the team to implement the features using agile coding methods. In addition, we held weekly meetings to record the progress of all team members and difficulties encountered.

2.2.4 Review (Finish coding)

When an iteration reaches its deadline, we would schedule a review meeting to evaluate the results and effectiveness of the features completed in this iteration to determine if optimizations, feature improvements, etc. should be included in the next iteration. For the parts that have been implemented perfectly, we add the period (date) it took to complete them to the product backlog for subsequent evaluation.

2.3 Third Phase: The Project Closure

Our airport kiosk users (or testers) are in constant communication with the Scrum master and developers throughout the process to refine the product features and their robustness. After the Outline Planning and Architectural Design phase and the Sprint Cycle phase over the past few months, we have now developed the completed functionality to satisfy all of our users' requirements. The project then entered the closure phase where we completed the required documentation and wrap up the project.

2.4 Risk Management

We carried out detailed risk assessment management in our projects. Firstly, we listed the project, product and business risks that were likely to arise during the process and assess their probability of occurrence, their consequences and their impact on our entire project and team. We also drew on some of Agile's strengths for risk management. For example, Agile advocates bringing risky requirements forward the schedule to have more time to assess the risk of the item and to identify feasible solutions[2]. Afterwards, we set aside more time for possible risks to emerge.
Throughout the process, some risks did arise (e.g. the design did not meet the requirements very well), but by assessing the risks in advance, we were able to overcome them well and continue working.

2.5 Quality Management

High quality software is characterized by ease of use, ease of modification and understanding of other developers' code. To ensure the quality of the software, in each iteration the front-end programs will be checked for quality by the back-end team from the customer's point of view and then feedback will be given. The back-end applications will be checked for quality by the front-end team from the front-end developer's point of view.

3.Requirements

3.1Applying the Requirements Finding Technique

3.1.1 Background reading

In order to understand the real needs of users, we did background investigation to have a deep understanding of the real situation and further understood the needs of users. We did some case studies, including check-in software from other airline company. We also analyzed the documentation of existing systems. We've captured requirements from a macro perspective for a smart flight check-in system.

3.1.2 Interviewing

We interviewed some some roles of people to better uncover the requirements. We conducted the survey using a combination of face-to-face and online interviews. Our interview strategy is to start with open questions and move on to more specific ones.
Here we list part of our interviewing questions:

- What do you want to be able to do in the check-in system?
- How would you like to log in to the check-in system?
- Do you want to pay for the seat selection and meal selection together or separately?

3.1.3 Observation

We observed at the check-in systems of some Chinese airlines, such as China Eastern Airlines. Through on-the-spot observation, we learned about the basic functions of the check-in system and the style of the user page. Based on the above observations, we have distilled three basic requirements that the programs we develop need to satisfy.
First, we should provide users with a login page that can log in in multiple ways to meet the needs of different users.
In addition, the system should support basic functions such as seat selection and meal selection, and support users to pay for service upgrades.
Last but not least, users can print boarding passes and luggage tags directly in the system without having to go to the counter to check.

3.1.4 Questionnaires

When understanding stakeholder needs, we use questionnaires to obtain more accurate needs without bias and conflict. Our questionnaires cover the basic functions of the software and how to use it. We carefully selected the responses to the questionnaire to prevent biased responses from misleading our design.

3.1.5 Requirements

In the specific division of requirements, we divide into functional requirements and non-functional requirements.

(1) Functional requirements

- Users can check-in use three methods
- Users can select the seat
- Users can upgrade the seat
- Users can select the meal set
- Users can upgrade the meal set
- Users can print their boarding pass
- Users can pay for the service
- Users can get a tag from the kiosk which would attach to the carry on baggage

(2) Non-functional requirements

- Users should receive error message if their input is invalid
- User should operate the software without training
- Use Java to develop the software as a stand-alone application
- Users can operate the software smoothly
- Design a flexible and extensible software

3.2 User Story

3.2.1 Epics and User Stories

In addition to capturing users requirements through the above techniques, we also use "User Stories". We split the epic into smaller user stories. Finally, we set acceptance criteria.(See in backlog)

3.2.2 Priorities of User Stories

We divided stories into must-have, should have, could have, and want to have. And we used MoSCoW rules to make the prioritization order. So we get an ordered list of our user stories. (See in backlog)

3.2.3 Estimation of User Stories

Story points are used to estimate the story.(See in backlog)

3.2.4 Iteration Planning

During the entire development iteration process, we basically followed the suggested timeline to complete each iteration process.(See in Appendix A)

3.2.5 Adaption to Changes

Because of the existence of risks, we were constantly making adjustments during the development process. Although we couldn’t meet each other during the epidemic, we used some online communication tools to often communicated with each other about some problems encountered in development and solve them in time. For example, we initially planned to use the icon to simulate the seat method to select the seat by clicking on the icon. Later, we found that javafx could not realize such a function, and finally changed it to use the choice-box instead .

3.2.6 Prototyping

In order to get user feedback quickly, we designed product prototypes.(See in Appendix B)

4 Analysis and Design

4.1 Class Analysis

4.1.1 Entity, Boundary and Control Classes identification

Entity Class

Entity Classes are mainly used to model long-lived and persistent information.[1]

We have three main Entity Classes, written in Customer.java, Seat.java, and CreditCards.java.

- We save the basic customer information in Customer_information.csv and extract it to memory via Customer.class as needed.
- Likewise, Seat.class and CreditCards.class extract the seat information and the "user-bank number correspondence information" from the corresponding files as required.

Controller Class

Controller classes are used to encapsulate control and coordination of the main actions and control flows. Our control classes can be divided into two main categories

- We completed many method classes for making changes to specific types of data, such as SeatSelector.java.
- In addition, we use javafx to complete the development, which is in MVC mode, with each page corresponding to a controller.

Boundary Class

Boundary classes are mainly responsible for rendering our user interfaces, listening to user actions and helping to complete interactions with them.

To achieve the requirements, we wrote 10 pages for login, seat selection, food selection, payment, etc. As mentioned before, we use the MVC pattern and the boundary parts are mainly implemented using a javafx class (PageApplication.java), 10 .FXML files and many .css files.

4.1.2 Reusability

To make our codes succinct and clear, we need to reduce the redundant codes as much as possible. Therefore, we extracted the identical process methods from the original class and encapsulated them as a particular method to make them reusable. [6]

Our project needs to deal with plenty of files regarding customers' information and seats and credit card information. For example, we need to read and write many CSV files to obtain the seats' information and convert it to the correct form like "4A" many times. In order to not use read and write in each class, we design a reusable method parseSeat().

```
public int parseSeat(Character column) {
    return switch (column) {
        case 'A' -> 0;
        case 'B' -> 1;
        case 'C' -> 2;
        case 'D' -> 3;
        case 'E' -> 4;
        case 'F' -> 5;
        default -> 0;
    };
}
```

Figure 4 Example of Reusability

There are many more examples like this, where we encapsulate and reuse a large number of methods on both the front and back ends to accomplish data manipulation (like the example above), page jumps (PageController.change_page()), style changes, etc.. In addition, we have written a lot of abstract classes to reuse, which will be described in detail below. A lot of .css files are used to facilitate the reuse of boundary class code, too.

4.1.3 Adaptability to Changes

Our fight check-in system can adapt to many real-world changes.

First, our class is highly reusable. In the class diagram, we can see the VIPCustomer class is inherited from the Customer class, meaning that in the future, the company can add more user-level without modifying

the existing code, instead, just inheriting from Customer or VIPCustomer is suggested. Besides, we use the living system of other teams, but not developed it by ourselves. So, we can change the interface to adapt more colorful needs of the trainees and other users.
Second, we make the function of each class clearly visible. With a highly encapsulate method, the front-end can easily use and ask for improvement of the method from the back-end with no need to get into detail. Also, it means that we can easily find where to modify when the new demand comes.

4.2 Design of Software

For class design, please go to Appendix C for more detailed UML components.

4.3 Design Principles

4.3.1 Single Responsibility Principle (SRP)

The single responsibility principle is a computer programming principle that states that every module, class, or function[5] should have responsibility over a single part of the functionality provided by the software.
We first planned the different responsibilities of the different classes and separated them, and we showed this process in **4.1**.
In our project, each class is only responsible for its own function and other classes are not affected when other functions are changed.
For example, we created SeatSelector.java, a controller class that is responsible for seat selection related functions, and when we change the functions of this class, none of the other classes, functions, or functions will be affected.

```java
package com.example.demo1;

import ...

public class SeatSelector {
    /**
     * This is the class for selecting the seats.
     */
    private final int row = 35;
    private final int column = 6;
    private String path;
    /** This is the constructor */
    public SeatSelector(String flightNum) { this.path = "src/main/resources/flightSeat/" + flightNum + ".txt"; }

    /** Find all the seats' status ...*/
    public int[][] findSeatState() {...}
    /** Select the available seat */
    public void update(int[][] state) {...}
    /** Select the available seat */
    public void update(String selectedSeat) {...}
    /** Find all the remaining seats ...*/
    public ArrayList<String> findRemainSeats() {
        int[][] state = new int[row][column];
        ArrayList<String> available = new ArrayList<>();
```

Figure 5 Example of SRP

4.3.2 Open-Closed Principle (OCP)

Software modules (classes, methods, etc.) should be "open for extension" and "closed for modification". Abstraction is the key to realizing the ' Open-Close' principle, where changes are not allowed at the abstraction level while extensions are allowed at the implementation level. The design method allows new functions to be added as new classes, leaving existing code as much as possible unchanged. As a result, the function is adaptive to later updates or modifications. [7]
To illustrate, we may refer to our own system. Our main account is Customer.java, while we can build a VIP customer class to extend it and add more privileges, like a discount of the total price, to such customers without writing repetitive codes and having no impact on previous accounts.

```java
public class VIPCustomer extends Customer {
    private double discount = 0.85;
    public VIPCustomer(String bookingNum) {
        super(bookingNum);
    }

    public VIPCustomer(String lastName, String idNum) {
        super(lastName, idNum);
    }

    @Override
    public int getMoney() {
        money = (int) (discount * (extrafeeMeal+extrafeeSeat));
        return money;
    }
}
```

Figure 6 Example of OCP

4.3.3 Don't Repeat Yourself (DRY)

The Don't Repeat Yourself Principle [8] refers to the process of programming without writing duplicate code, abstracting the public parts and encapsulating them into tool classes or using "abstraction" classes to abstract the public things, so as to reduce the coupling of the code, which not only improves the flexibility, robustness and readability of the code, but also facilitates the later maintenance or modification.
For example, to avoid copying and pasting lots of codes for some general function, we design some abstract class like FileOperation to reuse the code.

```java
package com.example.demo1;

import java.io.*;
/**
 * Saved the operation on the file, support methods in Customer.java
 *
 */
public abstract class FileOperation {
    // Empty incoming files
    public static void clearFile(String filename) throws IOException {
        FileWriter fileWriter =new FileWriter(filename);
        fileWriter.write( str: "");
        fileWriter.flush();
        fileWriter.close();
    }
```

Figure 7 Example 1 of DRY

In addition, we use similar abstract classes when completing the Page Jump Control, such as the abstract class goBack with its inherited LogOut and CommonGoBack.

GoBack.java | LogOut.java | CommonGoBack.java | CheckInController.java | PageApplication.java

```java
 * goback button 的样式
 * @author Yuqi Liu (youyang617@outlook.com)
 * @date 2022/5/26
 */
public abstract class GoBack {
    public GoBack(String title, String header, String content, Button button) {
        Alert alert = new Alert(Alert.AlertType.CONFIRMATION);
        alert.setTitle(title);
        alert.setHeaderText(header);
        alert.setContentText(content);

        if (alert.showAndWait().get() == ButtonType.OK){
            PageController controller = new PageController();
            try {
                controller.change_page(button);
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
```

Figure 8 Example 2 of DRY

To avoid copying code and then pasting them every time, in addition to using abstract classes, we also use approaches such as method classes, as we wrote in **4.1.2**.

4.3.4 Dependency Inversion Principle (DIP)

The Dependency Inversion Principle can be stated as: High-level modules should not depend on low-level modules; instead, both should depend on abstractions； Abstractions should not depend on details; instead, details should depend on abstractions.
To implement this principle, we use a number of abstract classes, just like we wrote in **4.3.3 DRY**

4.3.5 Liskov substitution principle (LSP)

According to the Liskov Substitution Principle(LSP)[9], in our design, we should ensure that the new derived classes generated when extending classes do not replace the functions in the old classes. To satisfy this principle, we adopt the form of inheritance.
In our code, VIP is an extension of Customer, where a change in the function of VIP does not affect the function of Customer. LogOut and CommonGoBack are related to GoBack in the same way.

```
package com.example.demo1;

public class VIPCustomer extends Customer {
    private double discount = 0.85;
    public VIPCustomer(String bookingNum) { super(bookingNum); }

    public VIPCustomer(String lastName, String idNum) { super(lastName, idNum); }

    @Override
    public int getMoney() {
        money = (int) (discount * (extrafeeMeal+extrafeeSeat));
        return money;
    }
}
```

Figure 9 Example of LSP

5.Implementation and testing

5.1 Implementation Strategy and Iteration Plan

5.1.1 Implementation Strategy

We separate our system into several micro and manageable subsystems. The two main branches are front-end and back-end. In each iteration, our target is to complete the subsystem separately. During this process, we assign two or more people to each subsystem project group for applying pair-programming. Before we start another build, we will guarantee that the previous parts are tested well through Juint. When all of these are finished, we will integrate them together as a usable version and then test it again by using the case and drawing its logic map.
We use Git for version control and as a remote platform for parallel development, which guarantees our working efficiency and the codes will never loss or damaged.
Our gitee link: https://gitee.com/you-yang/javafx_flightReservation

5.1.2 Iteration Plan

We use UML graphic to demonstrate the work after each iteration, containing finished subsystems, including the classes and their relations. The following table shows the details.

5.2 Testing （Strategy and Technique）

5.2.1 Strategy

Our aim is to verify the results from the implementation stage by testing each software build. We test the software in both component level and integration level.

(1) Which tests to run
Fifty percent of the tests will be autonomous and that includes unit testing and system testing. The rest of the test is done manually. While testing, we ensure that the non-test pages and functionality remain correct. More importantly, each topic use case should test its normal flow and behavior as well as two alternative flows and we try to make sure that our software works well under all kinds of incorrect input from users.
(2) The methods to run test
For unit testing based on system components, we use white-box testing to test the internal program logic. For system testing based on the whole system, we use black-box testing including partition testing and scenario-based testing to test the functional requirements of the software. Regression testing is used while integration testing.
(3) When to run testing
When it comes to the unit testing, it happens when each functional function is written during coding development. For the system testing, it happens after the project completed. As for integration testing, it's performed at each stage of iteration.
(4) Test standard
During the coding development, all the component functions can pass the unit testing so that we could build an integrated project. Even though unit tests have been implemented, errors can occur when the project is partially integrated, so we need system testing. At the same time, the software requirements of the project are also one of the standards.

5.2.2 Techniques

We use black boxes test to test the software requirements and use white box test to test the internal program logic.
5.2.2.1 Scenario testing
When the user enters our flight check-in kiosk, he would select seat, meal and pay if there is extra service and can also view his flight information, or modify the seat and meal. On the login page, he might type in the wrong user's name or ID number.
When the airline staff logs in, he can view the list of passengers and their check-in status for each flight.
5.2.2.2 Partition testing
We use it when we partition the whole program behaving in an equivalent way. There is a lot of input data in the login and registration screen, so we use partition tests to validate the input data.
5.2.2.3 Regression testing
The regression testing is used at each stage of iteration to ensure that the earlier functionality works after updating. After completing the calculate function of the extra money, we tested whether it could be successfully displayed on the screen after adding the relevant front-end pages.
5.2.2.4 Black-box test
We use partition test, scenario-based testing and regression testing to test the functional requirements for the whole system. Then whole test process will be put in Appendix D. Therefore, we only put an example here.

Test Case	Test Description	Cases	P/F	Bug.No	Bug	Comments
N/A	User modify	SetUp	--	--	--	Added to system
2.1	Test the modify of meal	2.1	P	0	0	When modify the meal,the type of meal in the database is changed
2.2	Test the modify of seat	2.2	P	0	0	When modify the seat,the type of seat in the database is changed.
2.3	Test the extra fee of spcial meal	2.3	P	0	0	The 50$ extrafee will be added if the meal is special type.
2.4	Test the extra fee of spcial seat	2.4	P	0	0	The 323$ extrafee will be added if the seat is special type.
2.5	Test the extra fee of normal meal	2.5	P	0	0	The 0$ extrafee will be added if the meal is normal type.
2.6	Test the extra fee of normal seat	2.6	P	0	0	The 0$ extrafee will be added if the seat is normal type.

Figure 10 Black Box Test Example of Modify

2.2.5 White box test
We use white-box test to test all statements and conditions. Here is one of the examples of our white-box test.Go to Appendix E to see the whole path.
Customer modify test: We use white-box test technology to test all the paths.

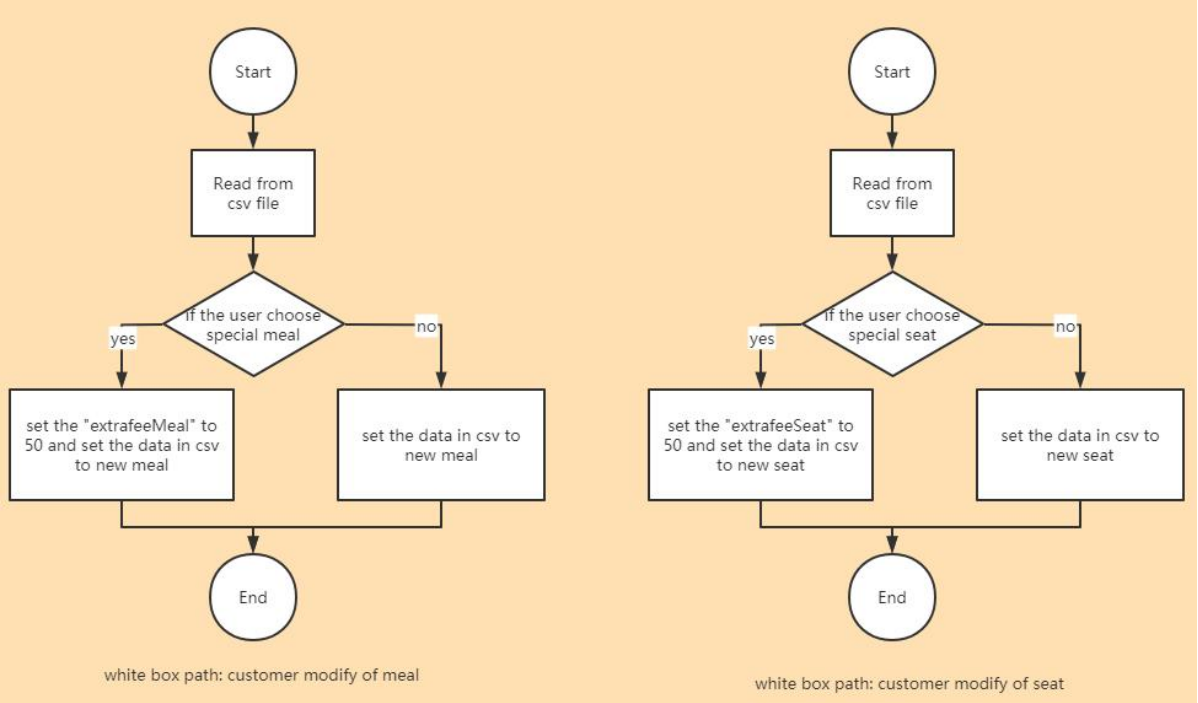


Figure 11 white box path of modify

5.3 The Using of TDD

We have experimented with Test Driving Development and have written Junit test classes for all our back-end classes, and this report will briefly describe the use of TDD in our development process, using the Customer class as an example.

Firstly, the Customer class is the most important class in the back-end system. This class stores all kinds of information about the customer and allows the program to query and modify the information in it.

We started by writing a Junit test program for the customer class, the search method needs to look up the corresponding passenger information in the database using the IdNum or BookingNum of the customer and return an array of strings containing all the information about the customer. The ModifyMeal and ModifySeat methods can to change the meal and seat information of the customer respectively. The code is shown in Appendix F.

Based on this test code, our group finally instant the Customer class and implemented the expected functionality. The final Junit test passed (please see the appendix F). The Customer class was delivered to the front end.

6. Reference

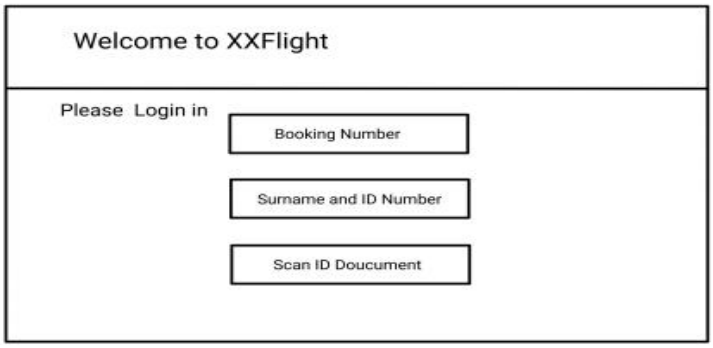
[1] "MoSCoW Analysis (6.1.5.2)". *A Guide to the Business Analysis Body of Knowledge* (2 ed.). International Institute of Business Analysis. 2009. [ISBN] 978-0-9811292-1-1
[2] A. Albadarneh, I. Albadarneh and A. Qusef, "Risk management in Agile software development: A comparative study," 2015 IEEE Jordan Conference on Applied Electrical Engineering and Computing Technologies (AEECT), 2015, pp. 1-6, doi: 10.1109/AEECT.2015.7360573.
[3]Gero, John S. (1990-12-15). "Design Prototypes: A Knowledge Representation Schema for Design". AI Magazine. 11 (4): 26. ISSN 0738-4602.
[4]"Software Prototyping - INGSOFTWARE". ingsoftware.com. Retrieved 2018-06-27.
[5]Martin, Robert C. (2003). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall. p. 95. ISBN 978-0135974445.
[6] "Clean Code: A Handbook of Agile Software Craftsmanship" by Robert. C. Martin.
[7] "Refactoring: Improving the Design of Existing Code" by Martin Fowler
[8] Leavens, Gary T.; Naumann, David A. (August 2015). "Behavioral subtyping, specification inheritance, and modular reasoning". *ACM Transactions on Programming Languages and Systems*. **37** (4): 1–88. doi:10.1145/2766446. S2CID 1379166.
[9] “Head First Object Oriented Analysis & Design” textbook by Brett McLaughlin et al

Appendix A:Iteration plan

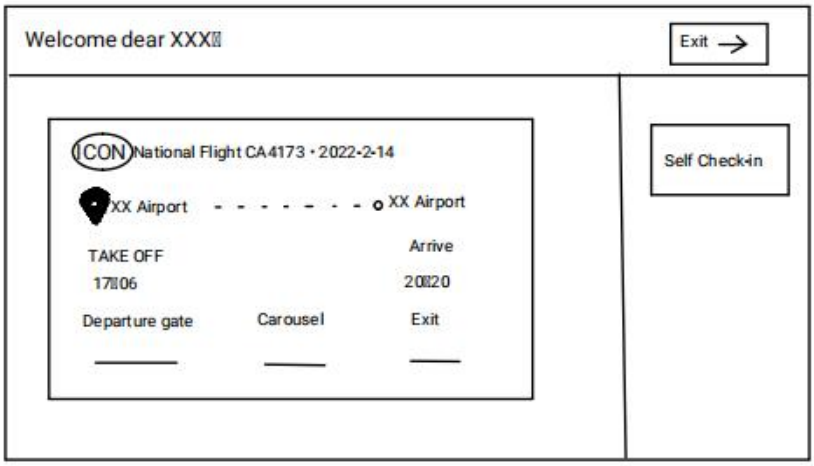
Iteration	Subsystem	Class	Component	Due
1	Basic UI frame;	Page, pageInController	Buttons, checkBox and label, goto_next, change_page, NewPage, MakeChoice;	31-Mar
	Account	Class Customer	ModifySeatNum, search.	
	CheckIn	CheckIn	initialize, CheckIn, class InputCheck, initText, classIssueEntries;	
	Check the validation of Input	InputCheck	checkOrderNumber, checkCustomerName,	
			checkIDNumber	
2	UI	SeatSet	class SeatController, getChoice, seatContinueClick, getSeat, getMyChoice, setMyChoice. (with interaction with data);	14 April
	Select the seats	SeatSelector; FileOperation	ISeatSelector: findSeatState, update, findRemainSeats,	
3	Print the boarding pass	BaggageController; BaggageLabelsController	Borading	28 April
	Print passenger list	Baggage; Check Passengers list;	initialize(URL location, ResourceBundle resources)	
4	ExpControl	Class ExpController	goBack, goNext, setSeat;	24 April
	UI refine	No new class	UI: Switching login methods by listening and jumping through Tabs component, clear the original input text when switching Tabs.	
	check Credit Card	Class CheckCreditCard:	check the validation and existence.	
5		Refine the codes	-	27-May

Appendix B:Prototyping

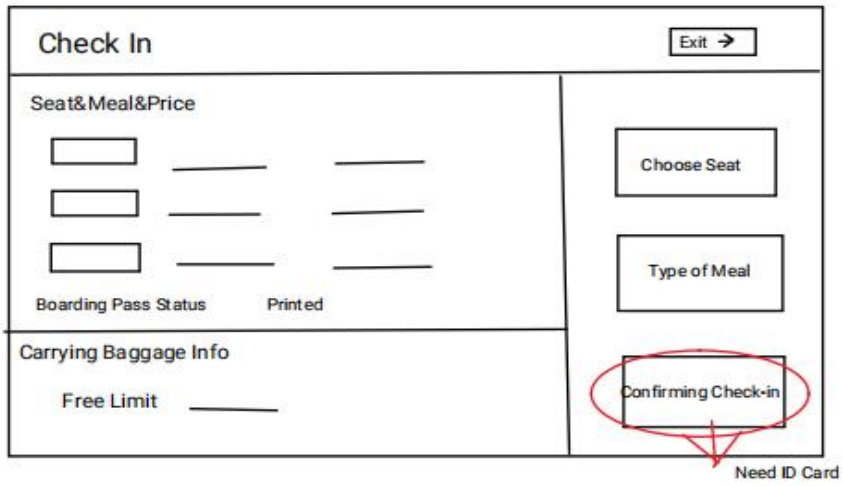
1.Welcome Page



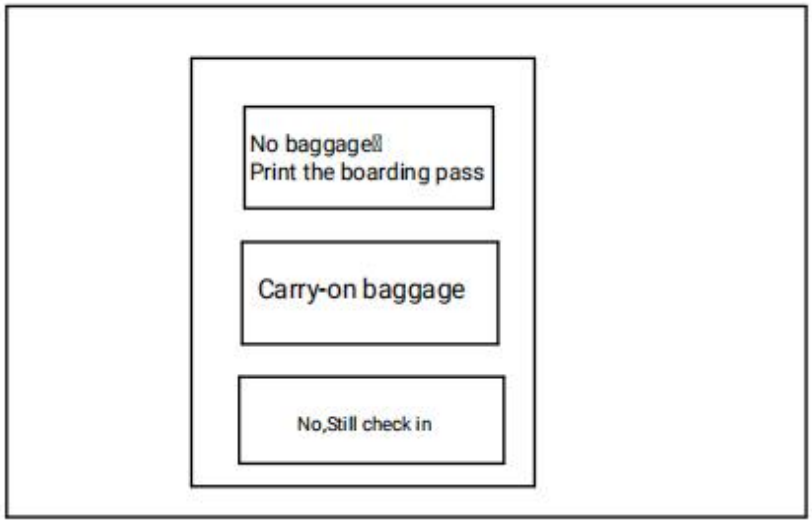
2.Summary booked flight



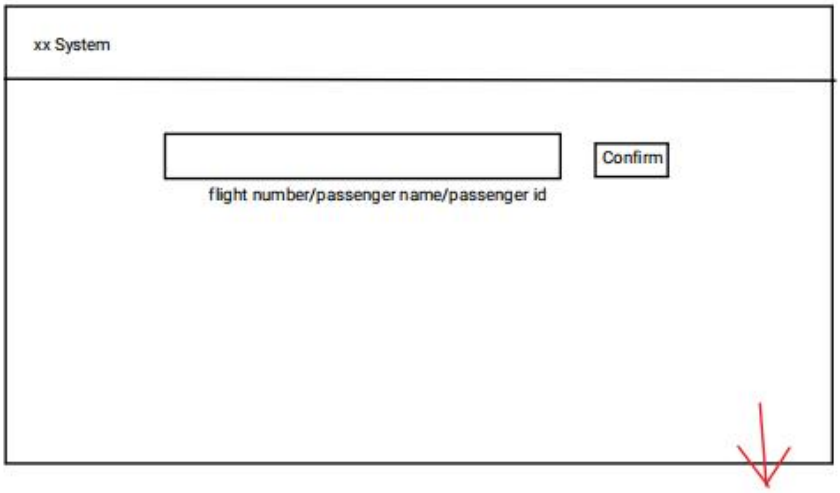
3.CHECK IN



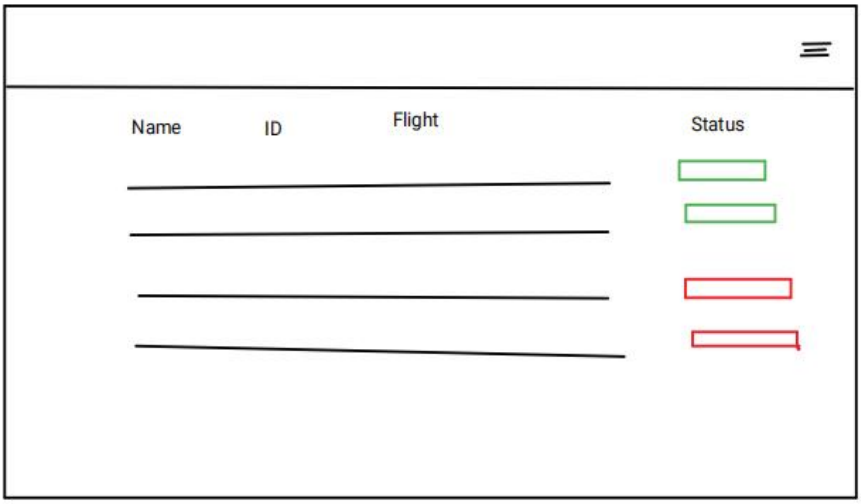
4 CARRYING BAGGAGE&PRINT BOARDING PASS



5 Passengers list

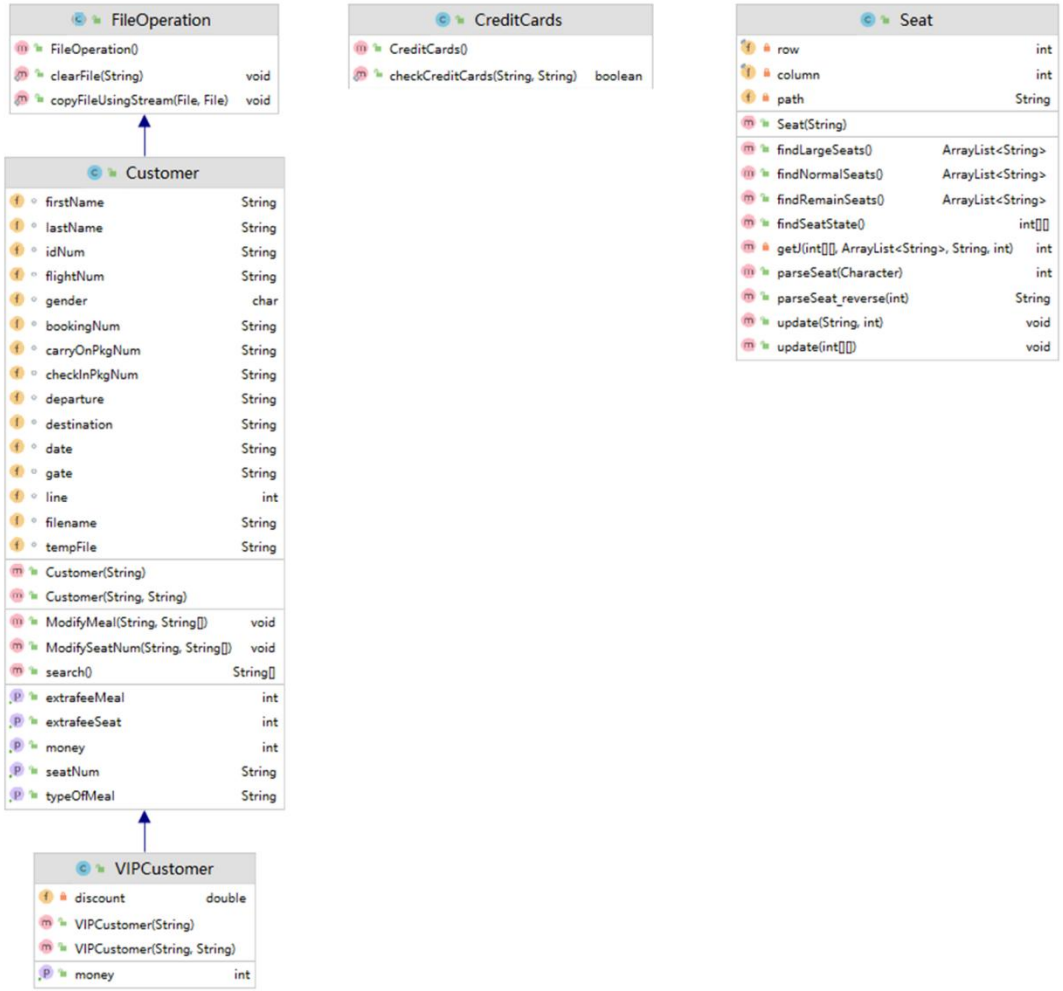


5 Passengers list

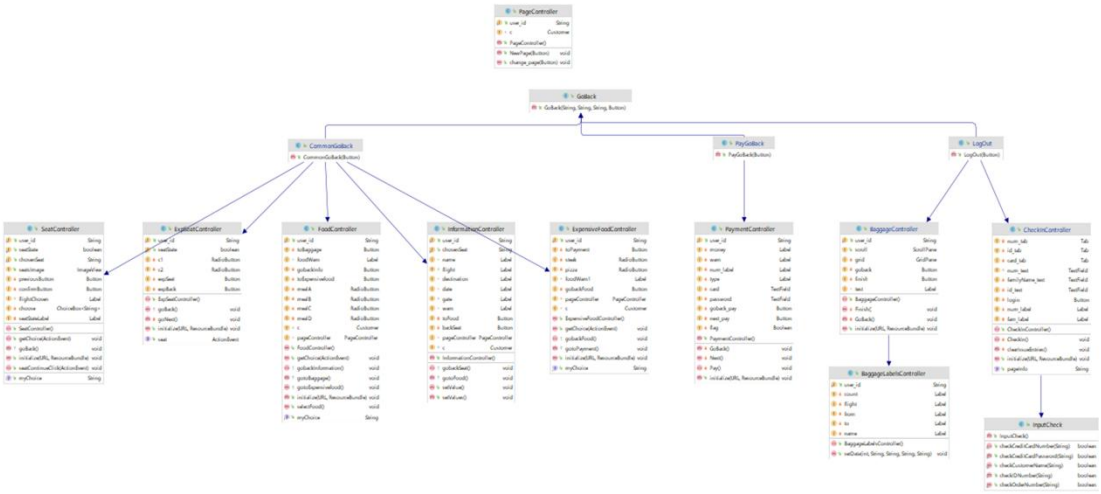


Appendix C:UML components

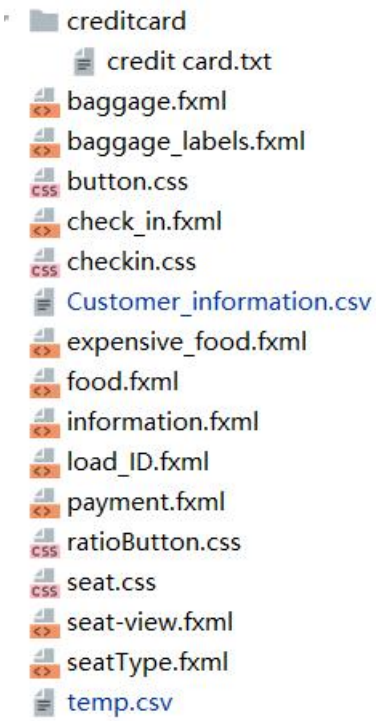
Entity class



Controller class



Boundary class

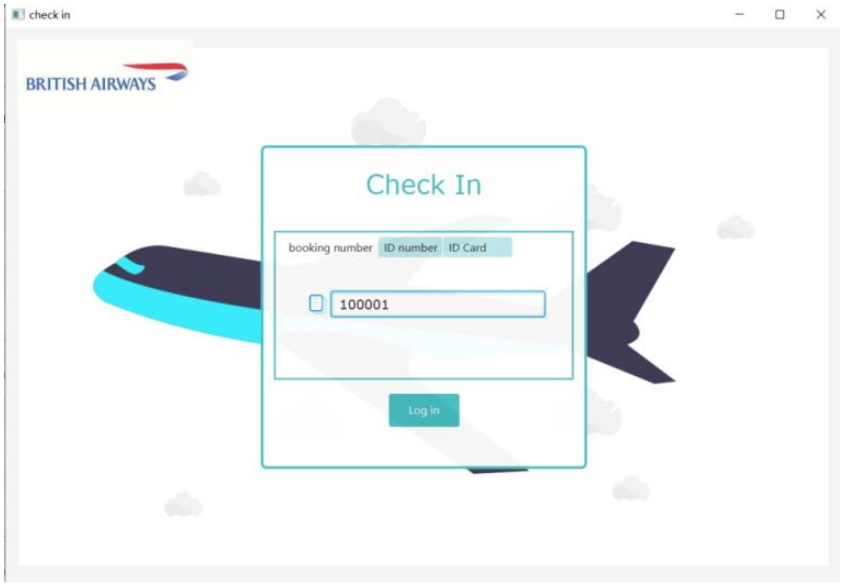


Appendix D:Black Box Test(with Screenshots)

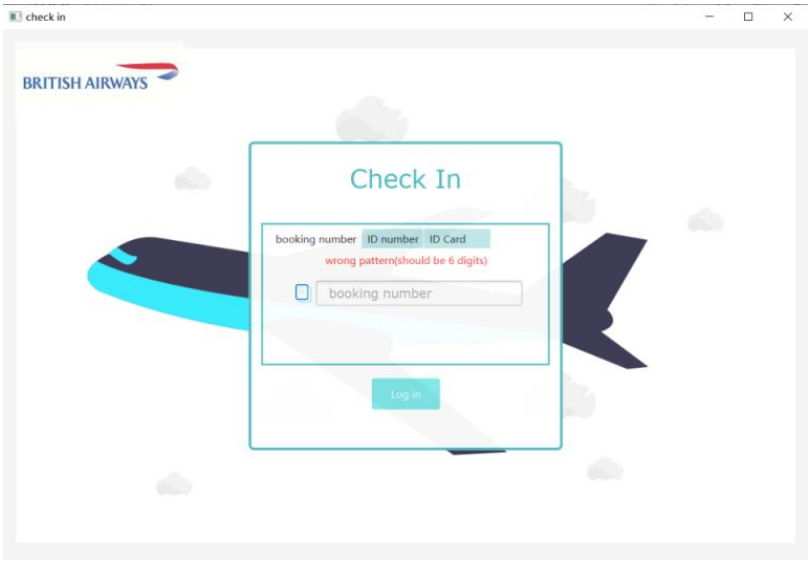
Test for login:

Test Case	Test Description	Cases	P/F	Bug.No	Bug	Comments
N/A	User login	SetUp	--	--	--	Added to system
1.1	Test the correct input	1.1	P	0	0	Expected result
1.2	Test the empty input	1.2	P	0	0	Expected result
1.3	Test the invalid input name	1.3	P	0	0	Expected result
1.4	Test the invalid input ID number	1.4	P	0	0	Expected result
1.5	Test the invalid order number	1.5	P	0	0	Expected result
1.6	Test the name not exsisting	1.6	P	0	0	Expected result

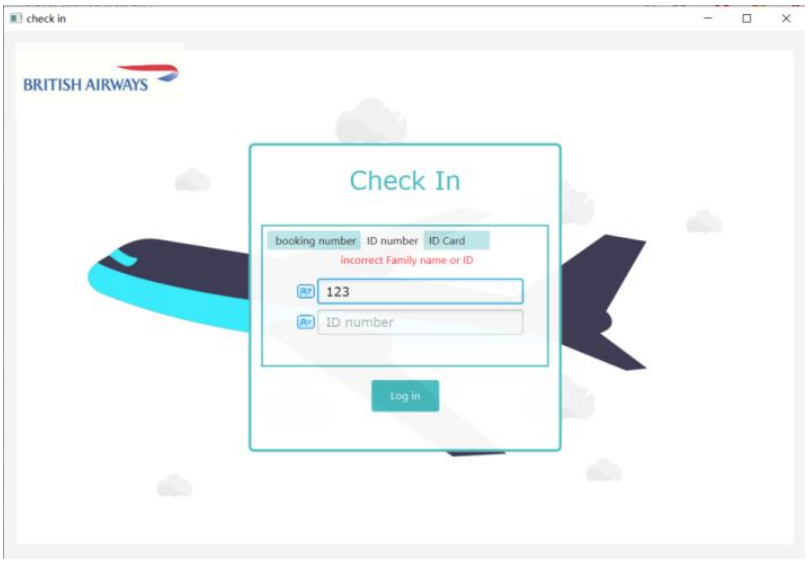
1.1



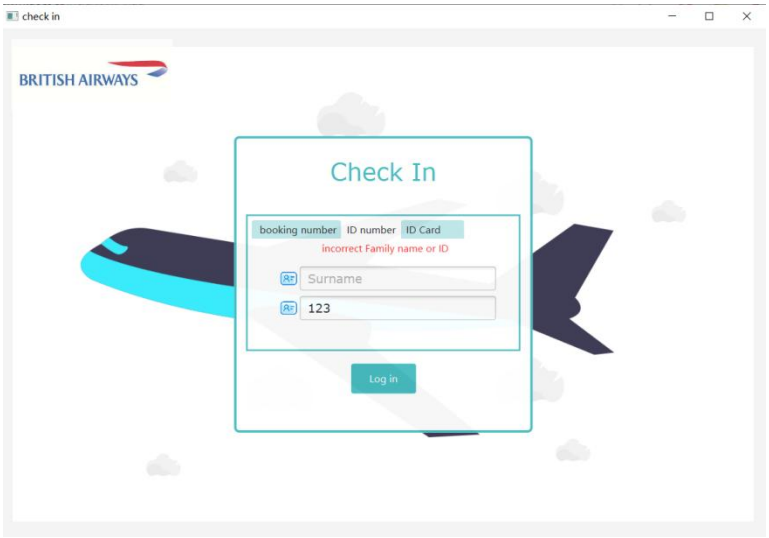
1.2


check in
BRITISH AIRWAYS
Check In
booking number
ID number
ID Card
wrong pattern(should be 6 digits)
booking number
Log in

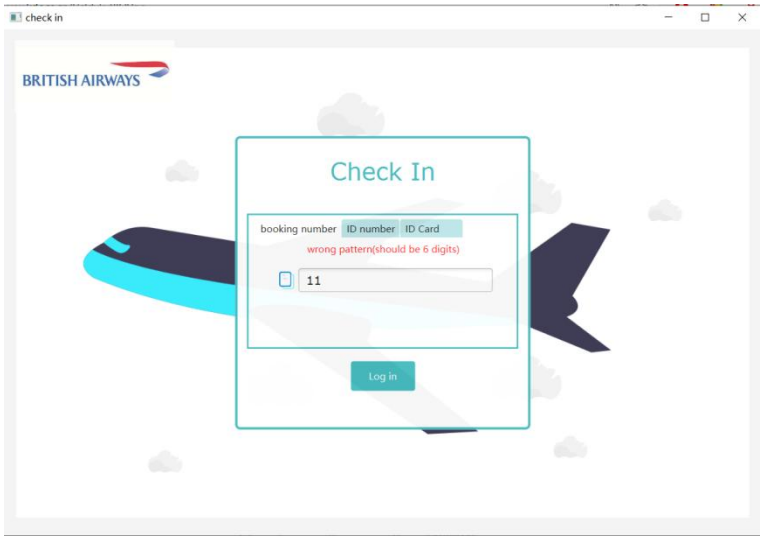
1.3


check in
BRITISH AIRWAYS
Check In
booking number
ID number
ID Card
incorrect Family name or ID
123
ID number
Log in

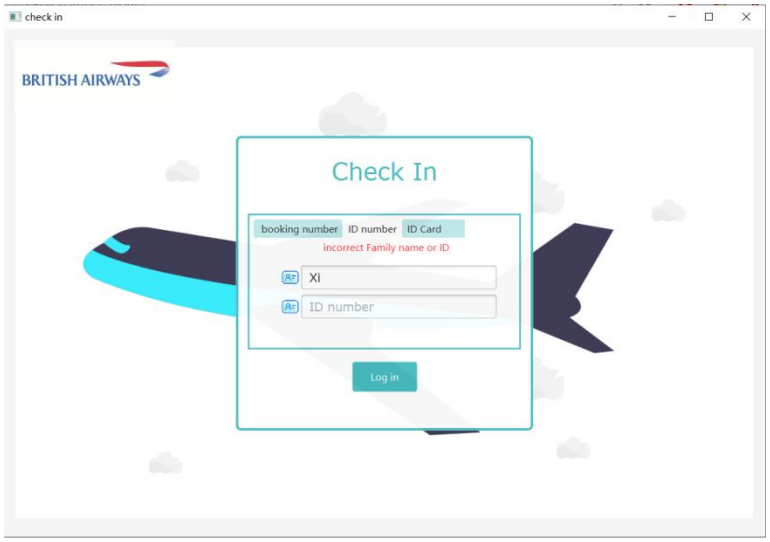
1.4


check in
BRITISH AIRWAYS
Check In
booking number
ID number
ID Card
incorrect Family name or ID
Surname
123
Log in

1.5


check in
BRITISH AIRWAYS
Check In
booking number
ID number
ID Card
wrong pattern(should be 6 digits)
11
Log in

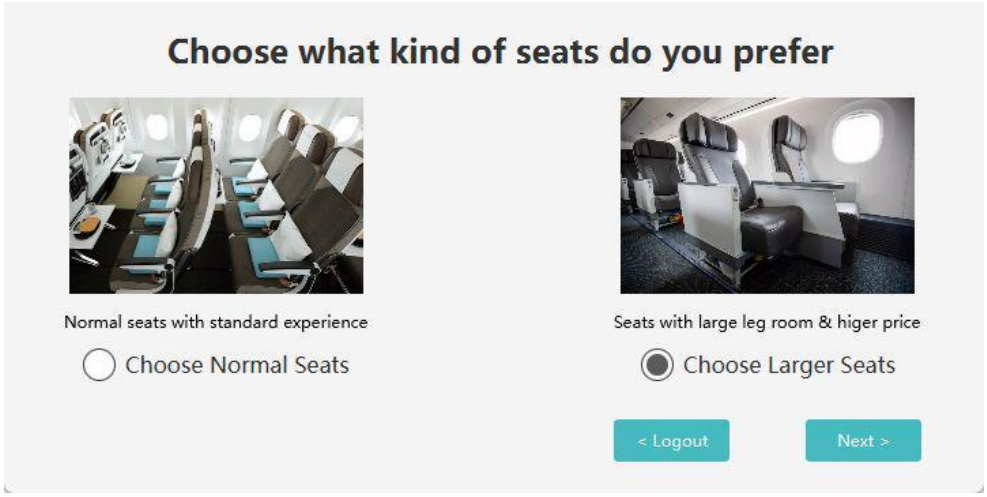
1.6


check in
BRITISH AIRWAYS
Check In
booking number
ID number
ID Card
incorrect Family name or ID
XI
ID number
Log in

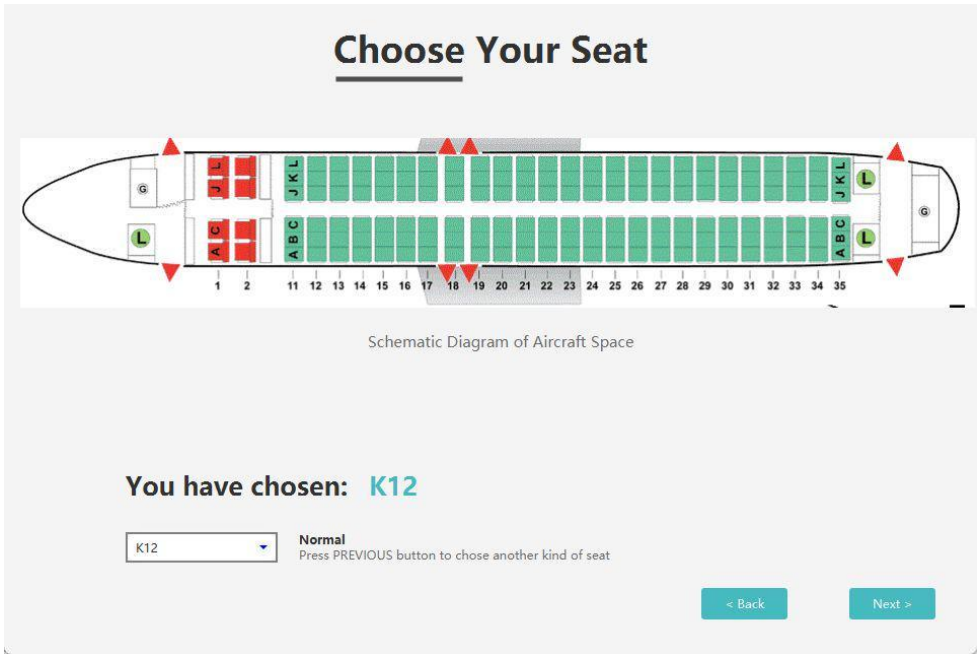
Test for Seat Selection:

Test Case	Test Description	Cases	P/F	Bug.No	Bug	Comments
N/A	Seat Selection	SetUp	--	--	--	Added to system
1.1	Test the choice of seat type	1.1	P	0	0	Expected result
1.2	Test the selection of specific seat	1.2	P	0	0	Expected result
1.3	Test the empty input	1.3	P	0	0	Expected result

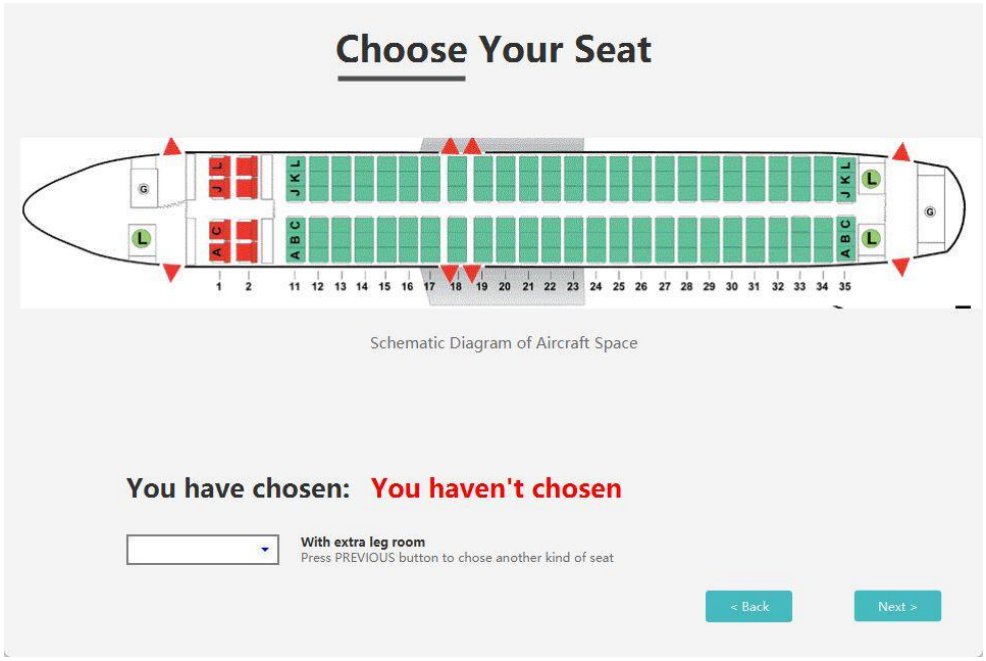
1.1



1.2



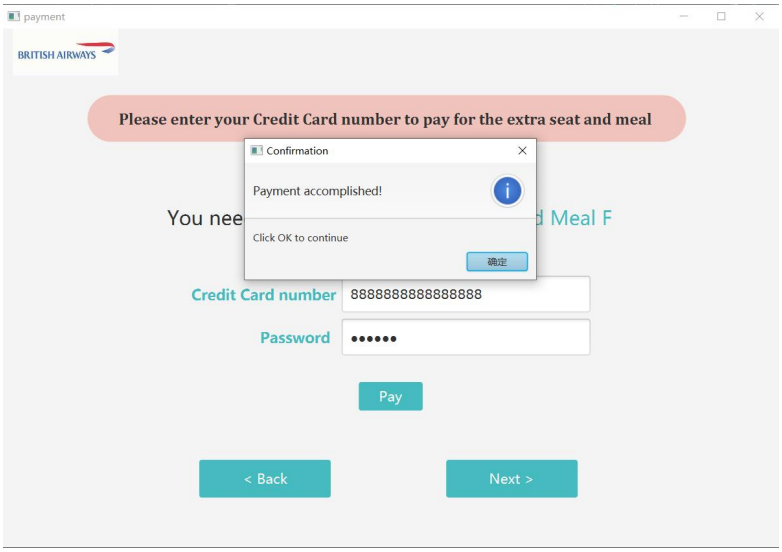
1.3


Choose Your Seat
G
L
J K L
A B C
1 2 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35
Schematic Diagram of Aircraft Space
You have chosen: You haven't chosen
With extra leg room
Press PREVIOUS button to chose another kind of seat
< Back
Next >

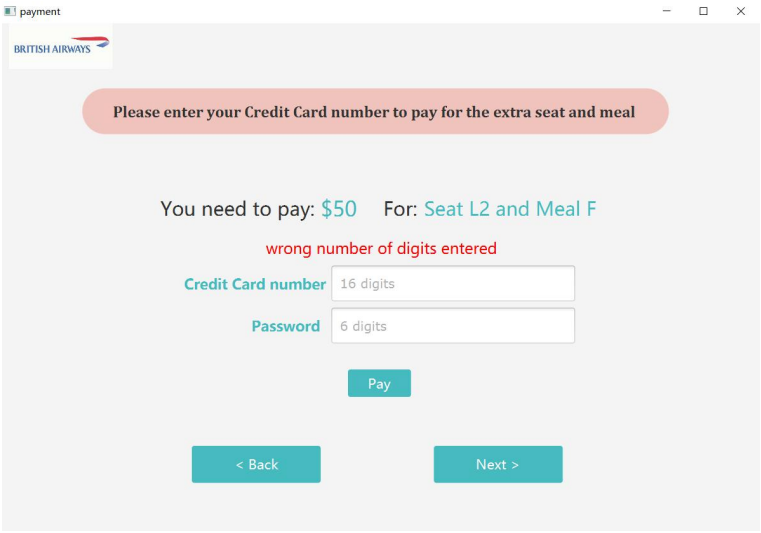
Test for payment:

Test Case	Test Description	Cases	P/F	Bug.No	Bug	Comments
N/A	Payment	SetUp	--	--	--	Added to system
1.1	Test the correct input	1.1	P	0	0	Expected result
1.2	Test empty input	1.2	P	0	0	Expected result
1.3	Test the invalid input number	1.3	P	0	0	Expected result
1.4	Test the invalid input password	1.4	P	0	0	Expected result
1.5	Test the unmatched input number and password	1.5	P	0	0	Expected result
1.6	Test the unfinished payment	1.6	P	0	0	Expected result

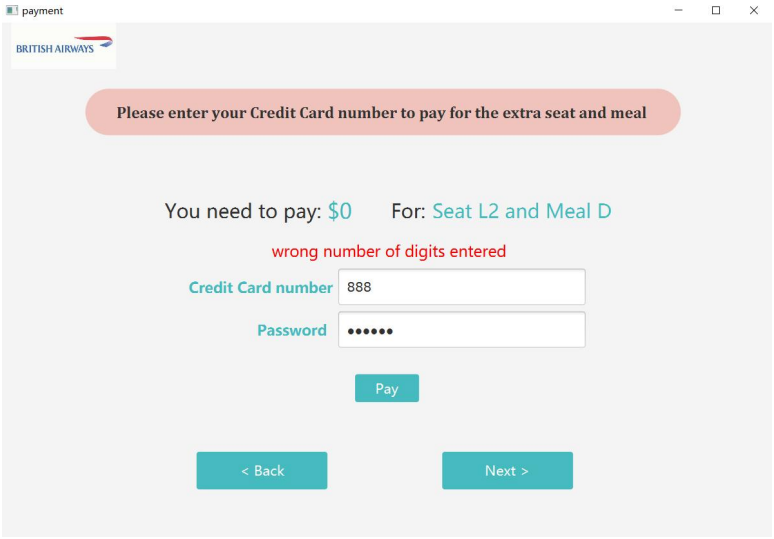
1.1



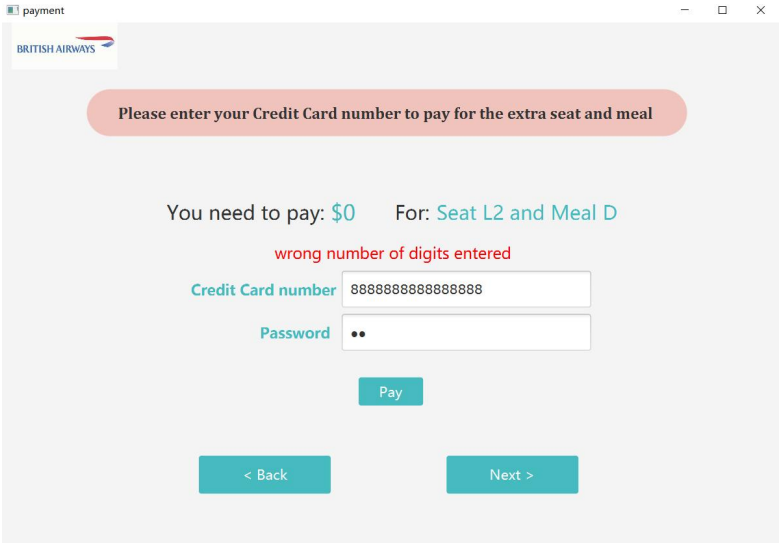
1.2



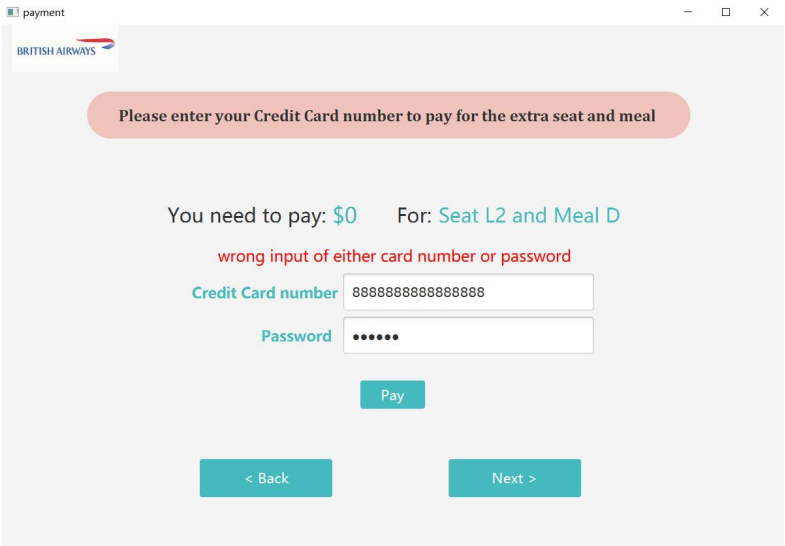
1.3


payment
BRITISH AIRWAYS
Please enter your Credit Card number to pay for the extra seat and meal
You need to pay: $0
For: Seat L2 and Meal D
wrong number of digits entered
Credit Card number
888
Password
Pay
< Back
Next >

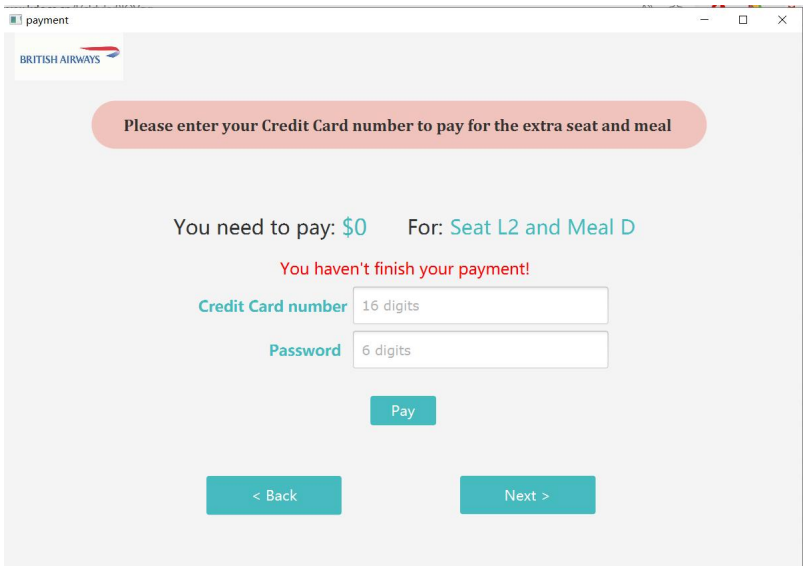
1.4


payment
BRITISH AIRWAYS
Please enter your Credit Card number to pay for the extra seat and meal
You need to pay: $0
For: Seat L2 and Meal D
wrong number of digits entered
Credit Card number
8888888888888888
Password
Pay
< Back
Next >

1.5


payment
BRITISH AIRWAYS
Please enter your Credit Card number to pay for the extra seat and meal
You need to pay: $0
For: Seat L2 and Meal D
wrong input of either card number or password
Credit Card number
8888888888888888
Password
Pay
< Back
Next >

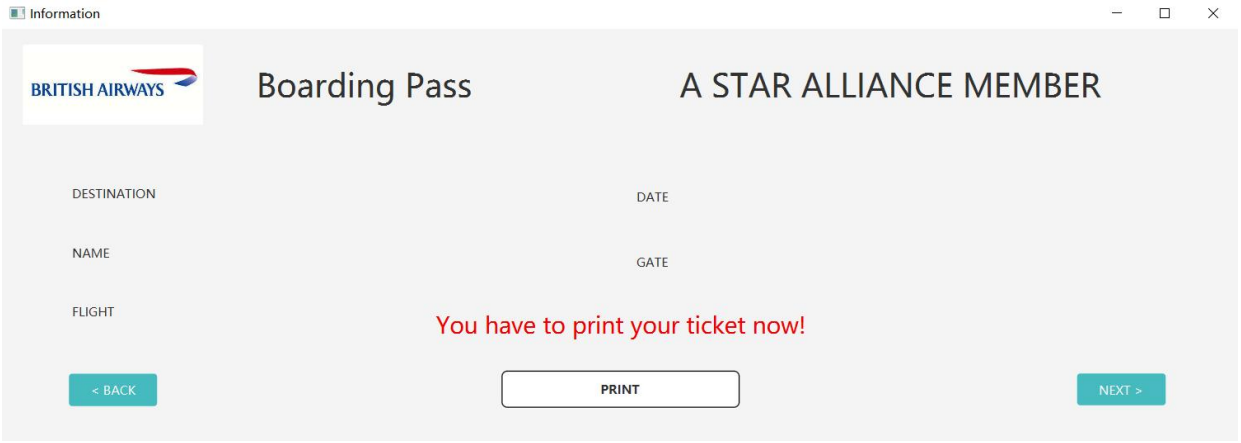
1.6


payment
BRITISH AIRWAYS
Please enter your Credit Card number to pay for the extra seat and meal
You need to pay: $0
For: Seat L2 and Meal D
You haven't finish your payment!
Credit Card number
16 digits
Password
6 digits
Pay
< Back
Next >

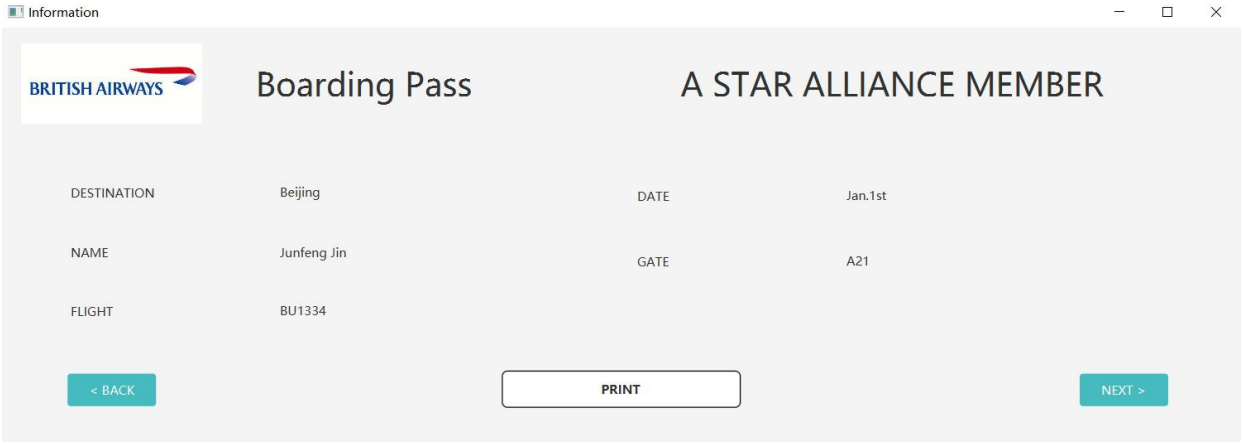
Test for Infomation:

Test Case	Test Description	Cases	P/F	Bug.No	Bug	Comments
N/A	information print	SetUp	--	--	--	Added to system
1.1	test print information	1.1	P	0	0	Expected result
1.2	test go next without printing	1.2	P	0	0	Expected result

1.1



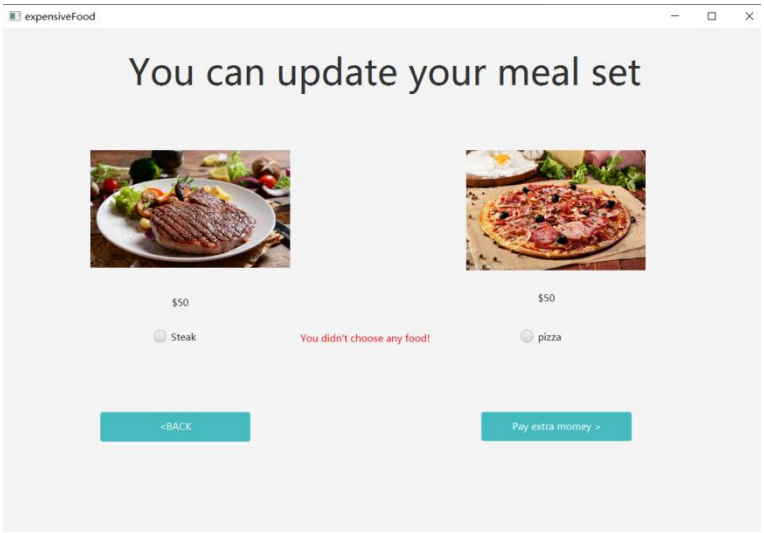
1.2



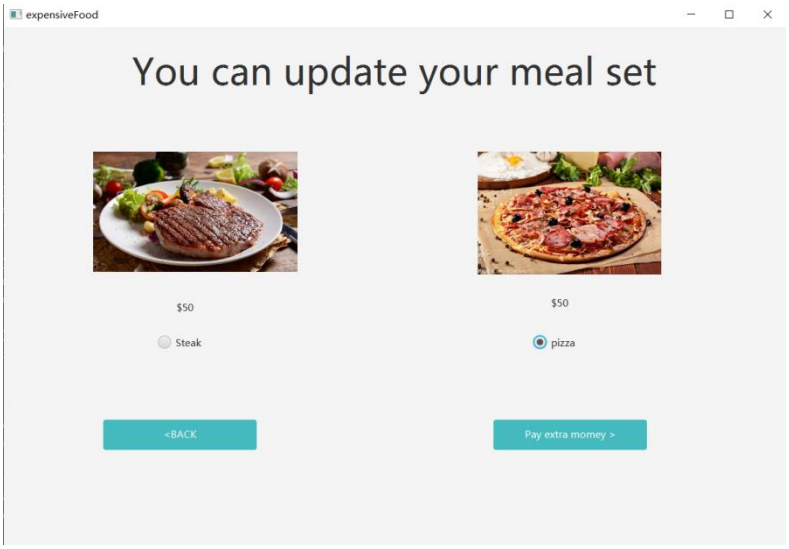
Test for select food:

Test Case	Test Description	Cases	P/F	Bug.No	Bug	Comment
N/A	food selection	SetUp	--	--	--	Added to system
1.1	Test select food	1.1	P	0	0	Expected result
1.2	Test empty input	1.2	P	0	0	Expected result
1.3	Test select food	1.3	P	0	0	Expected result
1.4	Test empty input	1.4	P	0	0	Expected result

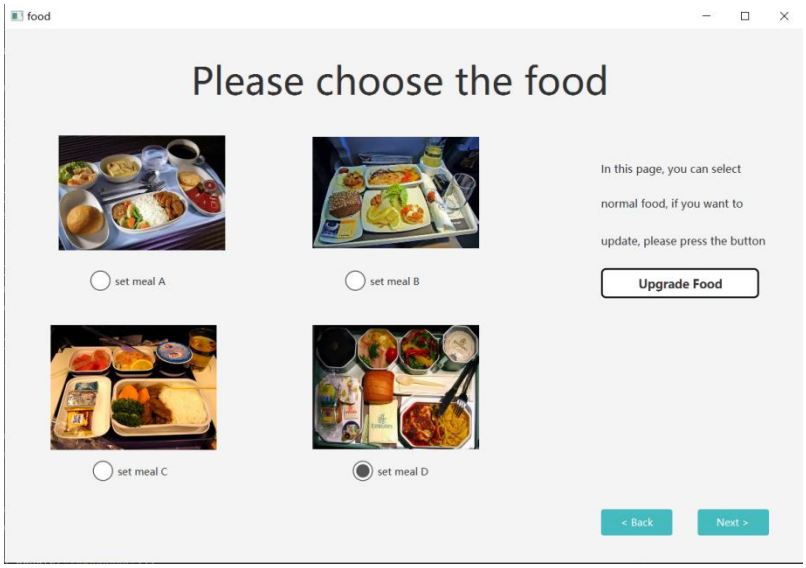
1.1



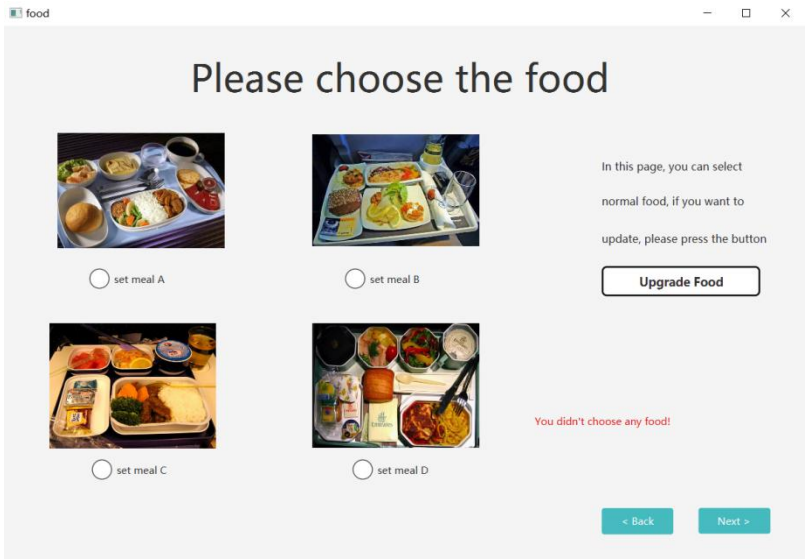
1.2



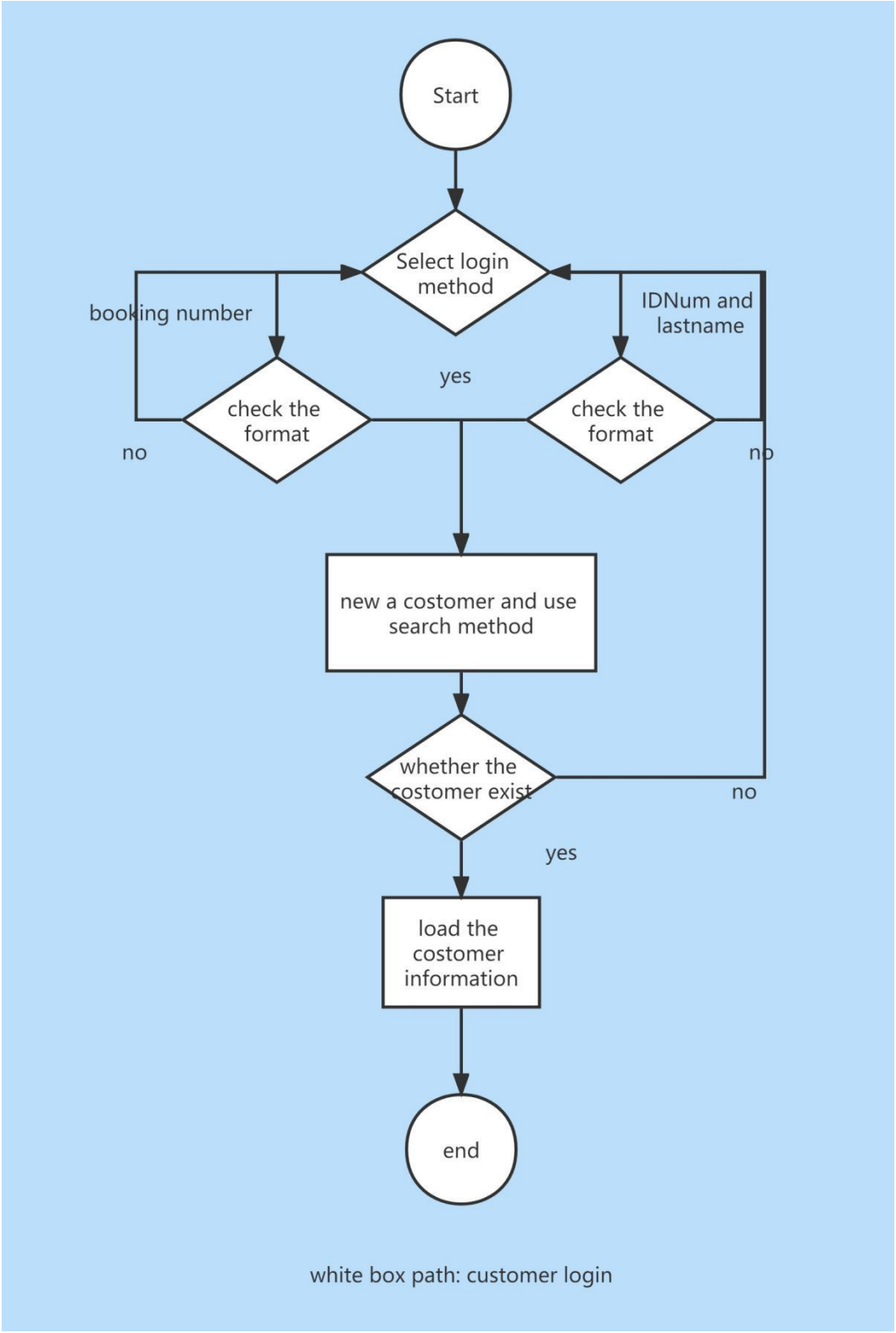
1.3


food
Please choose the food
set meal A
set meal B
In this page, you can select
normal food, if you want to
update, please press the button
Upgrade Food
set meal C
set meal D
< Back
Next >

1.4


food
Please choose the food
set meal A
set meal B
In this page, you can select
normal food, if you want to
update, please press the button
Upgrade Food
set meal C
set meal D
You didn't choose any food!
< Back
Next >

Appendix E:Whit Box Test



white box path: customer login

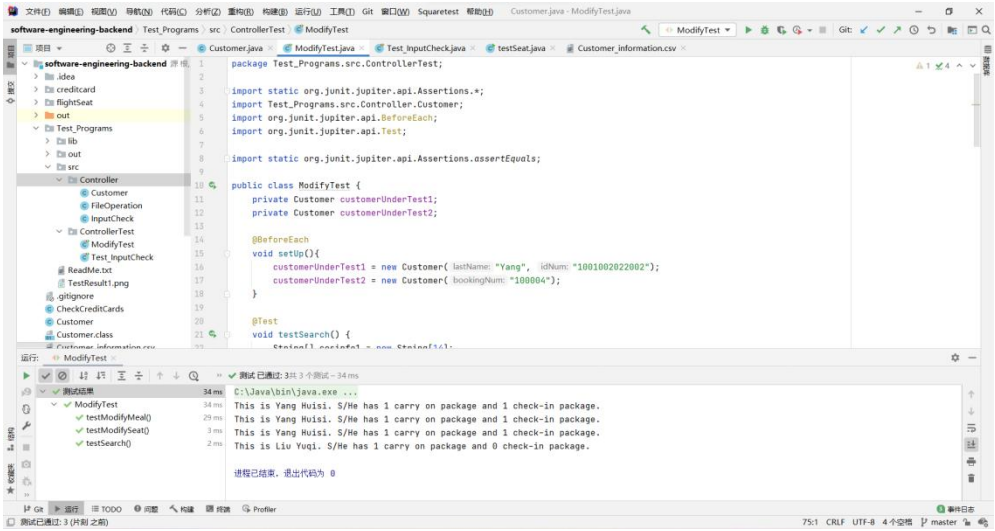
Appendix F:TDD

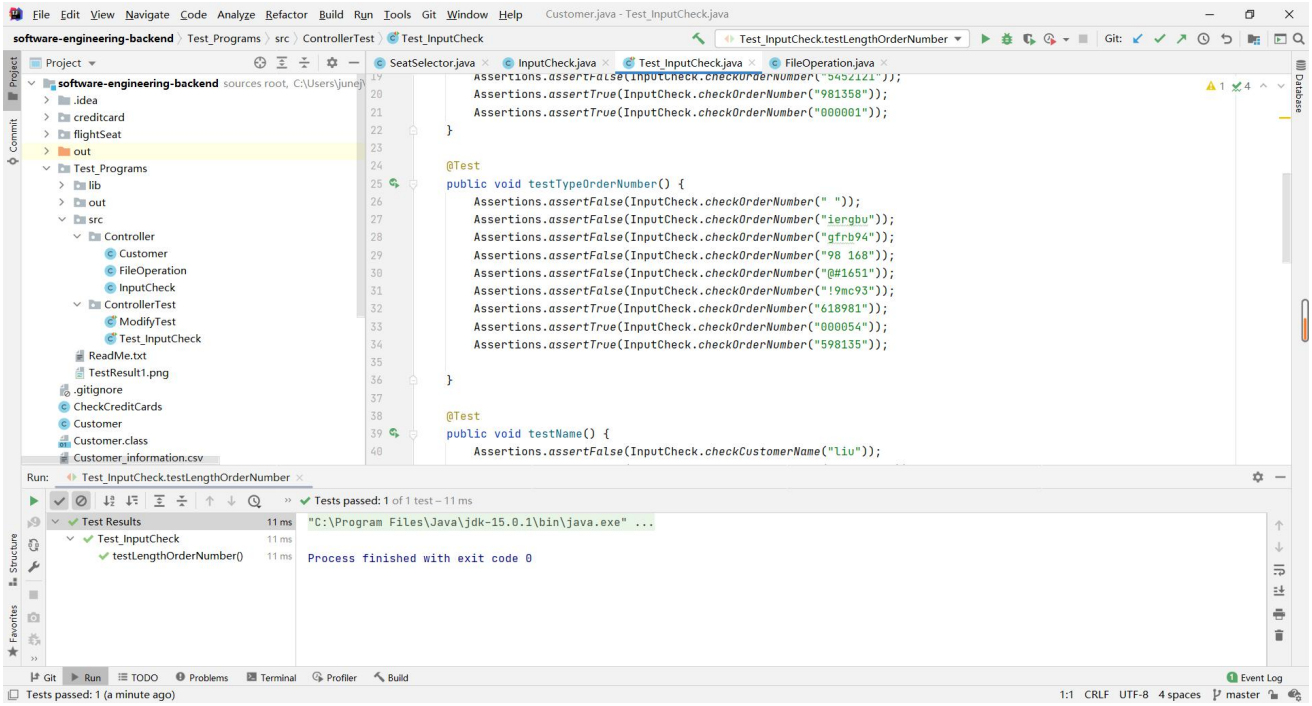
```
public class ModifyTest {
    private Customer customerUnderTest1;
    private Customer customerUnderTest2;

    @BeforeEach
    void setUp(){
        customerUnderTest1 = new Customer( lastName: "Yang",  idNum: "1001002022002");
        customerUnderTest2 = new Customer( bookingNum: "100004");
    }
        assertArrayEquals(cosinfo1, customerUnderTest1.search());
        assertArrayEquals(cosinfo2, customerUnderTest2.search());
    }

    @Test
    void testModifyMeal() {
        customerUnderTest1.ModifyMeal( typeOfMeal: "e",customerUnderTest1.search());
        assertEquals( expected: "e",customerUnderTest1.getTypeOfMeal());
        //test modify
        assertEquals( expected: 50, customerUnderTest1.getExtrafeeMeal());
        //test extra fee
    }

    @Test
    void testModifySeat() {
        customerUnderTest1.ModifySeatNum( SeatNum: "1A",customerUnderTest1.search());
        assertEquals( expected: "1A", customerUnderTest1.getSeatNum());
        assertEquals( expected: 323, customerUnderTest1.getExtrafeeSeat());
    }
```




File Edit View Navigate Code Analyze Refactor Build Run Tools Git Window Help
Customer.java - Test_InputCheck.java
software-engineering-backend
Test_Programs
src
ControllerTest
Test_InputCheck
Test_InputCheck.testLengthOrderNumber
Git:
Project
SeatSelector.java
InputCheck.java
Test_InputCheck.java
FileOperation.java
software-engineering-backend sources root, C:\Users\juney\
.idea
creditcard
flightSeat
out
Test_Programs
lib
out
src
Controller
Customer
FileOperation
InputCheck
ControllerTest
ModifyTest
Test_InputCheck
ReadMe.txt
TestResult1.png
.gitignore
CheckCreditCards
Customer
Customer.class
Customer_information.csv
Assertions.assertTrue(InputCheck.checkOrderNumber("981358"));
Assertions.assertTrue(InputCheck.checkOrderNumber("000001"));
}
@Test
public void testTypeOrderNumber() {
Assertions.assertFalse(InputCheck.checkOrderNumber(" "));
Assertions.assertFalse(InputCheck.checkOrderNumber("iergbu"));
Assertions.assertFalse(InputCheck.checkOrderNumber("gfrb94"));
Assertions.assertFalse(InputCheck.checkOrderNumber("98 168"));
Assertions.assertFalse(InputCheck.checkOrderNumber("@#1651"));
Assertions.assertFalse(InputCheck.checkOrderNumber("!9mc93"));
Assertions.assertTrue(InputCheck.checkOrderNumber("618981"));
Assertions.assertTrue(InputCheck.checkOrderNumber("000054"));
Assertions.assertTrue(InputCheck.checkOrderNumber("598135"));
}
@Test
public void testName() {
Assertions.assertFalse(InputCheck.checkCustomerName("liu"));
Run:
Test_InputCheck.testLengthOrderNumber
Tests passed: 1 of 1 test – 11 ms
Test Results 11 ms
Test_InputCheck 11 ms
testLengthOrderNumber() 11 ms
"C:\Program Files\Java\jdk-15.0.1\bin\java.exe" ...
Process finished with exit code 0
Git
Run
TODO
Problems
Terminal
Profiler
Build
Event Log
Tests passed: 1 (a minute ago)
1:1 CRLF UTF-8 4 spaces master